UNIVERSIDAD ANTONIO JOSÉ CAMACHO

Facultad de Ingeniería — Ingeniería de Sistemas

INFORME DE PROYECTO FINAL — PRIMER AVANCE

Ingeniería de Software III — FI303290

Nombre del Proyecto	GoPark — Plataforma web de gestión y control vehicular para parqueaderos
Integrantes del Equipo	Nicolás Agudelo Martínez — [Código] José García Cadena — [Código] Dilan Sneider García Bedoya — [Código]
Docente	Ing. José Lizardo Castro Guaraca
Período Académico	2026-2 — Segundo corte
Fecha de Entrega	30 / 10 / 2026

Tabla de Contenido

Tabla de Contenido....2
1. Introducción....4
2. Descripción del Sistema y Problema....4
2.1. Problema Identificado....4
2.2. Descripción del Sistema....5
2.3. Alcance del Proyecto....5
3. Stack Tecnológico....7
4. Diagrama de Arquitectura del Software....8
4.1. Diagrama de Contexto (Nivel 1 — C4)....8
4.2. Diagrama de Contenedores (Nivel 2 — C4)....8
4.3. Decisiones Arquitectónicas Clave....9
5. Diagrama de Despliegue....10
5.1. Descripción del Entorno de Despliegue....10
6. Metodología Scrum....11
6.1. Roles del Equipo....11
6.2. Ceremonias Scrum Realizadas....11
7. Product Backlog e Historias de Usuario....13
7.1. Product Backlog....13
7.2. Definition of Done (DoD)....14
8. Técnica de Priorización de Historias de Usuario....15
8.1. Técnica Seleccionada....15
8.2. Descripción de la Técnica....15
8.3. Aplicación al Proyecto....15
9. Cronograma de Sprints....16
9.1. Resumen de Sprints....16
9.2. Detalle por Sprint....16
Sprint 1....16
Sprint 2....17
Sprint 3....18
Sprint 4....18
9.3. Burndown Chart....18
10. Conclusiones y Lecciones Aprendidas....20
10.1. Conclusiones....20
10.2. Lecciones Aprendidas....20
Referencias....21
11. Anexos....21

Anexo A — Pantallas de Notion.....21
Anexo B — Pantallas de GitHub.....22
Anexo C — Pantallas del Sistema Funcional.....24

1. Introducción

GoPark es una plataforma web full-stack que moderniza el control vehicular en parqueaderos comerciales pequeños y medianos. Combina una interfaz de punto de venta (POS) ágil para los operadores de caja con un panel analítico en tiempo real para los administradores. El objetivo es eliminar las planillas en papel, reducir los tiempos de cobro y cerrar las fugas de dinero que se producen por cálculos manuales y por la falta de auditoría de los turnos de caja.

El proyecto se desarrolla en el curso Ingeniería de Software III (FI303290) de la Institución Universitaria Antonio José Camacho, período 2026-2, bajo el enfoque de Aprendizaje Basado en Proyectos. Lo construye un equipo de tres estudiantes con la metodología ágil Scrum, organizada en cuatro sprints de dos semanas. La gestión del backlog se lleva en Notion y el código se versiona en GitHub (repositorio niagma4/GoPark).

La motivación surge de una situación cotidiana en Cali: muchos parqueaderos siguen anotando placas a mano, calculan fracciones de hora con calculadora y cierran la caja sin un arqueo verificable. GoPark ataca estos problemas con cuatro capacidades centrales: check-in y check-out express con ticket QR, un motor automático de tarifas, control de caja por turnos (Cash Shifts) y un dashboard de aforo e ingresos.

Desde el punto de vista técnico, el sistema pone en práctica los resultados de aprendizaje del curso. Usa una arquitectura de monolito modular con frontend en Next.js y una API REST en NestJS sobre PostgreSQL, con autenticación JWT y control de acceso por roles. Incluye transacciones ACID con bloqueo pesimista para garantizar el aforo sin sobrecupos e integración y despliegue continuos con GitHub Actions hacia Vercel, Render y Supabase. Los principales desafíos son la concurrencia sobre el último cupo disponible, la exactitud del cálculo de tarifas en casos borde (minutos de gracia, redondeos y cruce de medianoche) y la curva de aprendizaje del stack.

Este documento corresponde al primer avance del proyecto, entregado en el segundo corte. Presenta el problema y el alcance, el stack tecnológico, la arquitectura (modelo C4) y el despliegue previsto. También describe la organización Scrum del equipo, el Product Backlog con su técnica de priorización y el estado real de los sprints al 30 de octubre de 2026, incluido el burndown. Cierra con las conclusiones, las lecciones aprendidas hasta ahora y los anexos de evidencia.

2. Descripción del Sistema y Problema

2.1. Problema Identificado

En Cali circulan en promedio 800.000 vehículos al día: 527.000 carros y 246.000 motocicletas (Alcaldía de Santiago de Cali, 2024a). Pese a esa demanda, la ciudad no tiene información consolidada sobre su oferta de parqueaderos: en 2024 la Secretaría de Movilidad abrió un censo para conocer los cupos, horarios y tarifas de los parqueaderos fuera de vía (Alcaldía de Santiago de Cali, 2024b).

Muchos parqueaderos pequeños y medianos todavía operan con fichas de papel o planillas, aunque la ley exige entregar al ingresar un recibo con la placa y la hora de ingreso (Ley 1801, 2016, art. 90) y el Estatuto del Consumidor admite medios tecnológicos para hacerlo (Ley 1480, 2011, art. 18). Los afectados son tres. Los administradores pierden dinero por errores al liquidar fracciones y por cierres de caja sin arqueo. Los operadores forman filas en horas pico. Los

clientes reciben comprobantes que se pierden. Resolverlo importa porque el negocio depende de la rotación de cupos y de un recaudo confiable.

2.2. Descripción del Sistema

GoPark es una aplicación web con dos actores internos y un rol de cliente. El Operador (rol OPERATOR) abre su turno de caja con una base de efectivo, registra entradas y salidas, cobra y cierra el turno con arqueo. El Administrador (rol ADMIN) configura tarifas y capacidad, audita los turnos y consulta las métricas. El cliente recibe un ticket QR al ingresar y lo presenta a la salida.

- **Autenticación y roles:** inicio de sesión con JWT en cookies HttpOnly y vistas diferenciadas por rol (RBAC).
- **Turnos de caja (Cash Shifts):** apertura con base inicial, bloqueo de operaciones sin turno abierto y cierre que compara el total esperado con el efectivo declarado.
- **Check-in express:** registro por placa normalizada y tipo de vehículo (carro, moto o bici) y emisión inmediata de un ticket con código QR.
- **Motor de tarifas y check-out:** liquidación automática con minutos de gracia, fracciones, hora completa y tarifa nocturna (patrón Strategy) y registro del método de pago.
- **Lector QR por cámara:** decodificación del ticket desde la cámara web en menos de un segundo.
- **Control de aforo:** reserva atómica del cupo por tipo de vehículo, sin sobrecupos ante accesos simultáneos.
- **Dashboard administrativo:** aforo en vivo, ingresos del día y la semana, horas pico y auditoría de turnos.

Flujo general: el operador abre turno → registra la entrada y entrega el ticket QR → a la salida escanea el QR (o digita la placa) → el sistema liquida el valor → registra el pago en el turno activo → al final de la jornada cierra el turno con arqueo, que el administrador audita desde el dashboard.

2.3. Alcance del Proyecto

La siguiente tabla delimita lo que se entregará al final del semestre.

En Alcance	Fuera de Alcance
Autenticación JWT y control de acceso por roles (ADMIN / OPERATOR)	Reservas de cupos en línea por parte de los clientes
Apertura, cierre y arqueo de turnos de caja	Pagos electrónicos integrados (pasarela, datáfono)
Check-in por placa y tipo de vehículo con ticket QR	Reconocimiento automático de placas por cámara (LPR/ANPR)
Motor de tarifas configurable y check-out con registro de pago	Facturación electrónica ante la DIAN
Lector QR por cámara web	Aplicación móvil nativa (iOS / Android)
Control de aforo concurrente por tipo de	Gestión de varias sedes en una misma

En Alcance	Fuera de Alcance
vehículo	cuenta (multisede)
Dashboard de ocupación, ingresos y horas pico	Integración con barreras o hardware de control de acceso
CI/CD, documentación técnica y despliegue en la nube	Mensualidades y abonados

3. Stack Tecnológico

Las tecnologías seleccionadas son las siguientes. Las versiones corresponden a las líneas estables previstas para el proyecto; el prototipo actual del repositorio (landing page) está construido con HTML5 y CSS3.

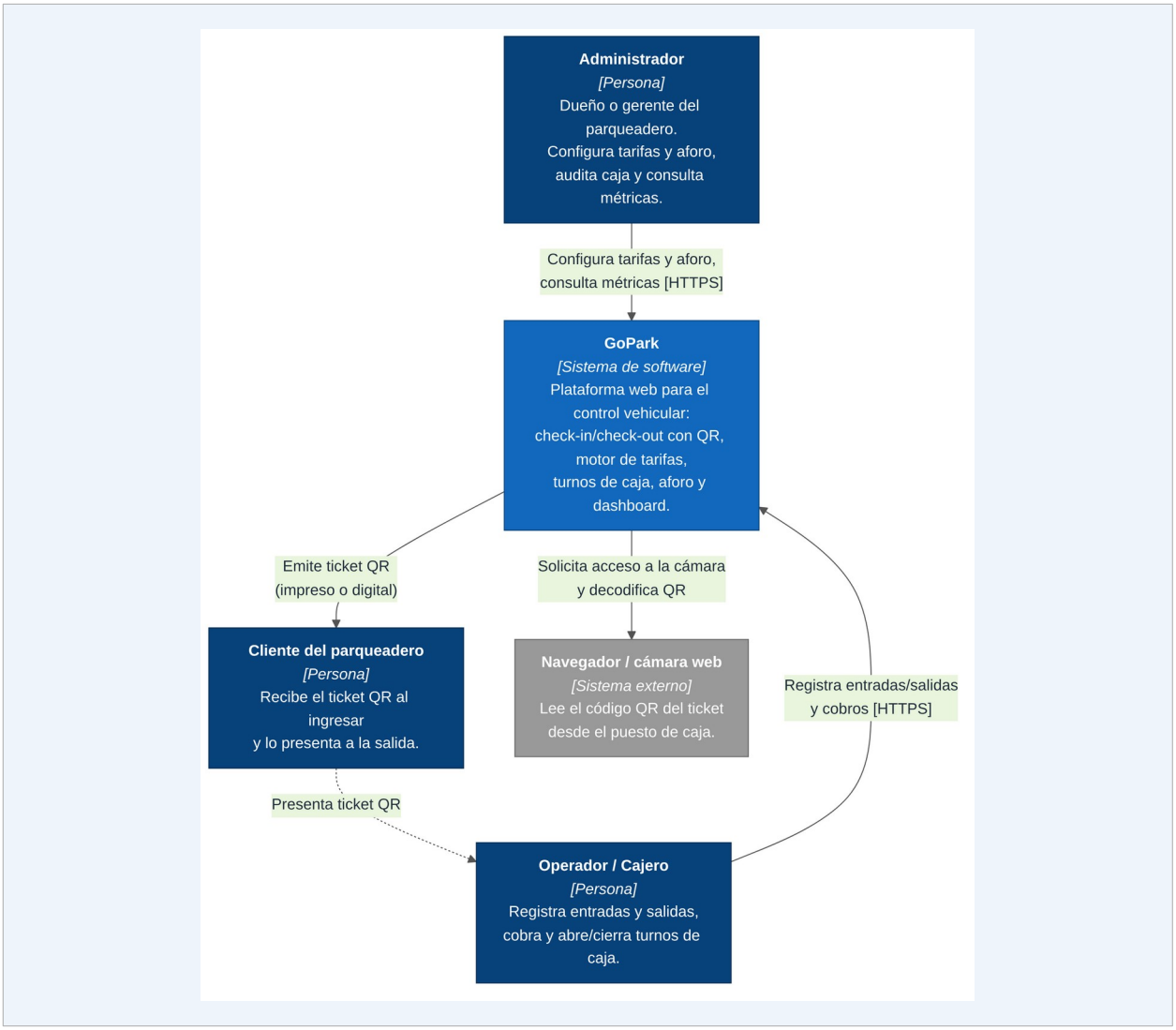
Categoría	Tecnología / Herramienta	Versión	Justificación
Lenguaje Backend	TypeScript sobre Node.js	5.x / Node 20 LTS	Un solo lenguaje en frontend y backend; tipado estático que reduce errores.
Framework Backend	NestJS (API REST)	10.x	Arquitectura modular (módulos, controladores, servicios), inyección de dependencias y guards para RBAC. El estilo REST es el visto en el curso.
Base de Datos	PostgreSQL	16	Integridad relacional, transacciones ACID y SELECT FOR UPDATE para el control de aforo.
ORM	Prisma ORM	5.x	Esquema declarativo, migraciones versionadas y cliente tipado.
Frontend	Next.js (React)	14.x / React 18	Componentes reutilizables, enrutamiento por archivos y despliegue nativo en Vercel.
UI	Tailwind CSS + shadcn/ui	3.x	Construcción rápida de una interfaz POS consistente y accesible.
Códigos QR	qrcode.react / html5-qrcode	—	Generación del ticket QR y lectura por cámara web desde el navegador.
Pruebas	Jest (+ Supertest)	29.x	Pruebas unitarias del motor de tarifas y pruebas de los endpoints de la API.
Control de Versiones	Git / GitHub	—	Colaboración por ramas y Pull Requests; historial de cambios del equipo.
Gestión de Proyecto	Notion	—	Tablero Scrum, Product Backlog e historias de usuario; el equipo ya trabajaba en Notion.
CI/CD	GitHub Actions	—	Lint, verificación de tipos (tsc) y pruebas en cada Pull Request; despliegue automático desde main.
Contenerización	Docker / Docker Compose	24.x	Entorno local idéntico para todo el equipo (web, API y base de datos).
Nube	Vercel + Render + Supabase	—	Capas gratuitas; SSL automático; PostgreSQL gestionado con backups.

4. Diagrama de Arquitectura del Software

La arquitectura se documenta con el modelo C4 en sus dos primeros niveles (Brown, s.f.). GoPark es un monolito modular: un frontend Next.js y una única API REST en NestJS organizada por módulos de dominio, que comparten una base de datos PostgreSQL.

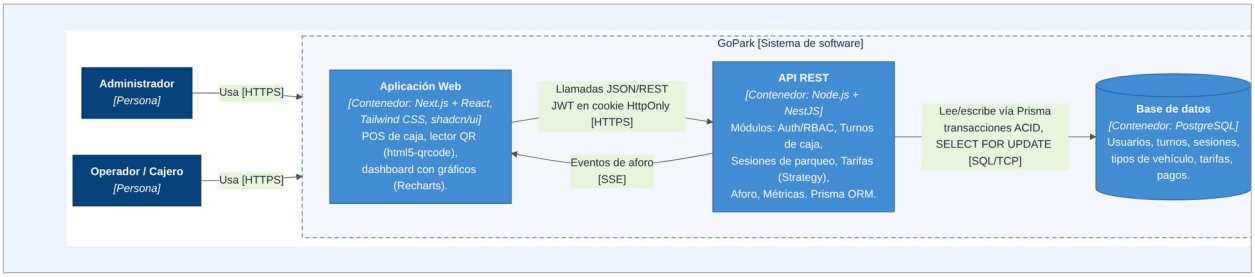
4.1. Diagrama de Contexto (Nivel 1 — C4)

GoPark se relaciona con tres personas (administrador, operador y cliente) y con un sistema externo: el navegador del puesto de caja, cuya cámara lee los códigos QR.



4.2. Diagrama de Contenedores (Nivel 2 — C4)

El sistema se compone de tres contenedores: la aplicación web (Next.js), la API REST (NestJS + Prisma) y la base de datos PostgreSQL. La web consume la API con JSON sobre HTTPS y recibe los cambios de aforo por Server-Sent Events.



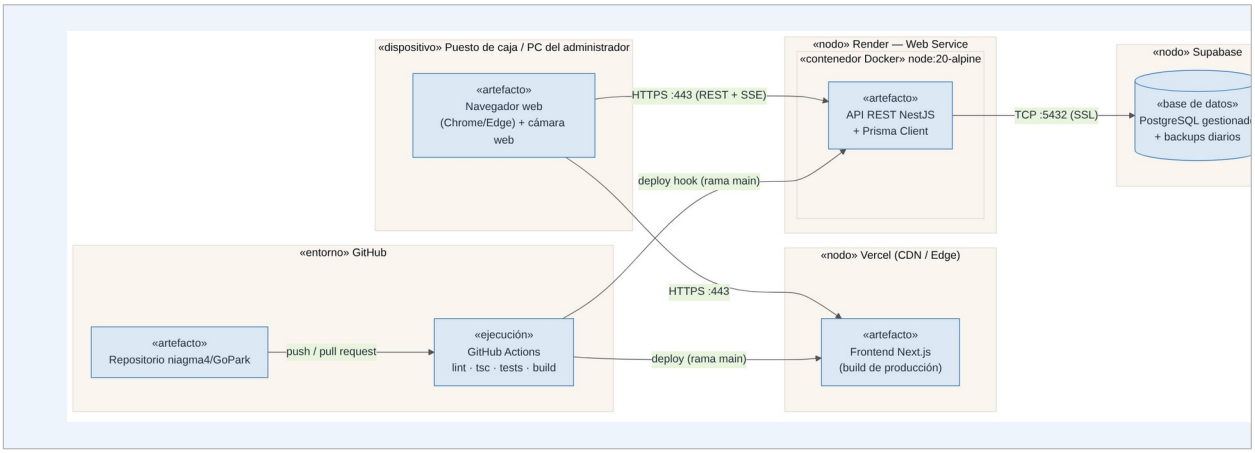
4.3. Decisiones Arquitectónicas Clave

Estas son las decisiones de arquitectura más relevantes tomadas hasta el momento.

Decisión	Alternativas Consideradas	Razón de la Elección
Patrón arquitectónico: monolito modular	Monolito modular vs. microservicios	Un equipo de tres personas y cuatro sprints no justifica la complejidad operativa de los microservicios. Los módulos (auth, turnos, sesiones, tarifas, aforo, métricas) quedan desacoplados y podrían separarse más adelante (Newman, 2021).
Estilo de API: REST	REST vs. GraphQL	Los recursos son claros (shifts, sessions, rates), el estilo REST es el estudiado en el curso y se documenta con OpenAPI/Swagger directamente desde NestJS (Fielding, 2000).
Persistencia: SQL (PostgreSQL)	SQL vs. NoSQL (MongoDB)	El dominio es transaccional (cobros, arqueos, cupos) y exige integridad referencial y transacciones ACID (Bass et al., 2021).
Concurrencia de aforo: bloqueo pesimista	SELECT FOR UPDATE vs. bloqueo optimista vs. aislamiento serializable	Garantiza que dos operadores no ocupen el último cupo a la vez; el volumen de escrituras es bajo, así que el costo del bloqueo es aceptable (PostgreSQL Global Development Group, s.f.).
Autenticación: JWT en cookie HttpOnly	Cookie HttpOnly vs. localStorage vs. sesión en servidor	Evita el robo del token por XSS y mantiene la API sin estado (OWASP Foundation, s.f.).
Motor de tarifas: patrón Strategy	Strategy vs. condicionales en el servicio	Cada tipo de vehículo tiene su estrategia de cálculo, que se prueba de forma aislada y se extiende sin modificar el código existente, según el principio abierto/cerrado (Gamma et al., 1994; Martin, 2017).

5. Diagrama de Despliegue

El despliegue previsto (Sprint 4) separa el frontend, la API y la base de datos en servicios gestionados con capa gratuita. Siguiendo las prácticas de entrega continua (Humble y Farley, 2010), GitHub Actions valida cada Pull Request y despliega automáticamente al hacer merge en main. Mientras tanto, el desarrollo se hace en local con Docker Compose.



5.1. Descripción del Entorno de Despliegue

Proveedor / Infraestructura	Vercel (frontend), Render Web Service (API en Docker) y Supabase (PostgreSQL). Desarrollo local con Docker Compose.
Sistema Operativo	Contenedor Linux node:20-alpine (API); Vercel y Supabase son servicios gestionados
Contenerización	Docker Compose — 3 servicios en local (web, api, db); Dockerfile de la API para Render
Base de Datos	PostgreSQL 16 — instancia gestionada en Supabase, con backups diarios
Dominio / URL de acceso	Pendiente (se publica en el Sprint 4). Local: http://localhost:3000 (web) y http://localhost:4000 (API)
Certificado SSL	Sí — automático (Let's Encrypt) provisto por Vercel y Render

6. Metodología Scrum

6.1. Roles del Equipo

Los roles siguen la Guía de Scrum (Schwaber y Sutherland, 2020). El equipo tiene tres integrantes; los tres participan como Developers y dos de ellos asumen además los roles de Product Owner y Scrum Master. El docente, Ing. José Lizardo Castro Guaraca, actúa como stakeholder y evaluador.

Rol Scrum	Integrante Asignado	Responsabilidades Principales	Justificación
Product Owner	Dilan Sneider García Bedoya	Mantiene y prioriza el Product Backlog en Notion, define criterios de aceptación y acepta las historias terminadas.	Lideró la definición de la propuesta de valor y de las historias de usuario de GoPark.
Scrum Master	Nicolás Agudelo Martínez	Facilita las ceremonias, hace seguimiento del tablero y elimina impedimentos.	Administra el repositorio de GitHub y coordina el flujo de trabajo del equipo.
Developer (x3)	Nicolás Agudelo Martínez, José García Cadena, Dilan Sneider García Bedoya	Diseño, codificación, pruebas, revisión de código e integración.	Todos participan en el desarrollo.

6.2. Ceremonias Scrum Realizadas

Las ceremonias se adaptaron al horario del curso (B-Learning) y a la disponibilidad del equipo.

Ceremonia	Frecuencia	Duración Aprox.	Descripción / Evidencia
Sprint Planning	Al inicio de cada sprint	1 hora	Selección de historias del backlog en Notion, estimación en story points y definición del objetivo del sprint.
Daily Scrum	Dos veces por semana	15 min	Virtual por WhatsApp/Discord: qué hice, qué haré y qué me bloquea.

Ceremonia	Frecuencia	Duración Aprox.	Descripción / Evidencia
Sprint Review	Al final de cada sprint	30–45 min	Demostración del incremento al Product Owner y retroalimentación del docente. Pendiente la del Sprint 1, que sigue abierto.
Retrospectiva	Al final de cada sprint	30 min	Formato «qué funcionó / qué mejorar / acciones», registrado en Notion. Pendiente al cierre del Sprint 1.

7. Product Backlog e Historias de Usuario

El Product Backlog contiene nueve historias de usuario y suma 58 story points (escala Fibonacci). Cada historia sigue el formato «Como [actor], quiero [acción], para [beneficio]».

7.1. Product Backlog

ID	Historia de Usuario	Actor	Criterios de Aceptación	Story Points	Prioridad
HU-01	Como administrador/cajero, quiero iniciar sesión con credenciales seguras y acceder a las vistas de mi rol, para proteger la operación y la información del negocio.	Administrador / Operador	JWT almacenado en cookie HttpOnly. El operador solo accede a caja/operación; el administrador, a métricas y configuración. Middleware de protección de rutas en frontend y API.	8	Alta
HU-02	Como cajero, quiero abrir un turno con una base de efectivo y cerrarlo al finalizar mi jornada, para rendir cuentas exactas de mi caja.	Operador	No se registran entradas ni cobros sin un turno abierto. El cierre compara el total esperado con el efectivo declarado.	5	Alta
HU-03	Como operador, quiero registrar el ingreso de un vehículo por placa y tipo y generar un ticket QR, para agilizar la entrada en horas pico.	Operador	La placa se normaliza (mayúsculas, sin espacios). No se admite una placa con sesión ACTIVE. El QR codifica el id o token de la sesión.	8	Alta
HU-04	Como operador, quiero escanear o digitar la placa para liquidar el valor y registrar el pago, para cobrar sin errores de cálculo.	Operador	Motor de tarifas desacoplado: minutos de gracia, fracción y hora completa. El pago (CASH/TRANSFER) se asocia al turno activo. La sesión pasa a COMPLETED.	8	Alta
HU-05	Como operador, quiero escanear el ticket QR con la cámara web, para cargar la salida al instante.	Operador	Decodificación en menos de 1 segundo. Carga de la sesión sin recargar la pantalla. Manejo de códigos	5	Baja

ID	Historia de Usuario	Actor	Criterios de Aceptación	Story Points	Prioridad
			inválidos o ya pagados.		
HU-06	Como administrador, quiero que el sistema bloquee el ingreso con el aforo lleno y evite sobrecupos simultáneos, para no exceder la capacidad.	Administrador	Validación atómica de cupos por tipo de vehículo. Con dos ingresos simultáneos al último cupo, uno es rechazado limpiamente.	8	Alta
HU-07	Como administrador, quiero ver el aforo en tiempo real, los ingresos y las horas pico, para tomar decisiones del negocio.	Administrador	Vehículos activos por tipo. Gráfico de ingresos del día/semana y tabla de turnos auditables.	8	Media
HU-08	Como evaluador técnico, quiero clonar el repositorio, ejecutar las pruebas y ver un demo en vivo, para validar la calidad del proyecto.	Evaluador técnico	README con arquitectura, decisiones y credenciales demo. Pipeline CI que ejecute lint y tests en cada Pull Request.	5	Media
HU-09	Como dueño de un parqueadero, quiero conocer la propuesta de valor de GoPark en una página informativa, para decidir si solicito una demo.	Visitante (dueño de parqueadero)	Secciones de solución, comparación y beneficios. Llamados a la acción «Solicitar demo» y «Prueba gratis».	3	Baja

7.2. Definition of Done (DoD)

Una Historia de Usuario se considera TERMINADA cuando cumple todos los siguientes criterios:

- El código fue revisado por al menos un compañero de equipo (code review).
- Las pruebas unitarias asociadas pasan exitosamente.
- La funcionalidad fue probada manualmente en el entorno de desarrollo.
- El código fue integrado a la rama principal sin conflictos.
- La historia fue aceptada formalmente por el Product Owner.
- La historia quedó registrada como «Done» en Notion con evidencia (captura o enlace al Pull Request).

8. Técnica de Priorización de Historias de Usuario

Para ordenar el backlog, el equipo combinó MoSCoW con una matriz de valor frente a esfuerzo.

8.1. Técnica Seleccionada

Técnica utilizada	MoSCoW, complementada con matriz Valor vs. Esfuerzo
Justificación	MoSCoW es sencilla de aplicar con el Product Owner y deja claro qué es indispensable para tener un producto funcional al final del semestre. La matriz Valor/Esfuerzo desempata las historias dentro de una misma categoría.
Herramienta de apoyo	Notion (propiedades «MoSCoW», «Valor» y «Story points» en la base de datos del backlog)

8.2. Descripción de la Técnica

MoSCoW, propuesta por Clegg y Barker (1994) dentro del método DSDM, clasifica los requerimientos en cuatro categorías. Must have reúne lo indispensable, sin lo cual el producto no sirve (registrar, cobrar y cuadrar caja). Should have agrupa lo importante, pero que admite una solución temporal. Could have contiene lo deseable si sobra capacidad, y Won't have lo que queda fuera de esta entrega. Dentro de cada categoría, cada historia recibe un valor de negocio de 1 a 5, definido por el Product Owner, y un esfuerzo igual a sus story points. Se programan primero las de mayor relación valor/esfuerzo y se respetan las dependencias técnicas (por ejemplo, sin autenticación y turnos no puede haber cobros).

8.3. Aplicación al Proyecto

Resultado de aplicar la técnica al backlog de GoPark (Won't have: reservas en línea, pagos electrónicos, LPR y facturación electrónica):

ID Historia	Historia de Usuario (resumen)	Categoría / Peso	Criterio de Priorización	Resultado
HU-01	Autenticación y roles	Must Have / Alta	Valor: 5, Esfuerzo: 8 (base técnica de todo)	P1 — Sprint 1
HU-02	Turnos de caja	Must Have / Alta	Valor: 5, Esfuerzo: 5	P2 — Sprint 1
HU-03	Check-in y ticket QR	Must Have / Alta	Valor: 5, Esfuerzo: 8	P3 — Sprint 2
HU-04	Tarifas y check-out	Must Have / Alta	Valor: 5, Esfuerzo: 8	P4 — Sprint 2
HU-06	Aforo	Must Have /	Valor: 4, Esfuerzo: 8	P5 — Sprint 3

ID Historia	Historia de Usuario (resumen)	Categoría / Peso	Criterio de Priorización	Resultado
	concurrente	Alta		
HU-07	Dashboard	Should Have / Media	Valor: 4, Esfuerzo: 8	P6 — Sprint 4
HU-08	CI/CD, docs y despliegue	Should Have / Media	Valor: 4, Esfuerzo: 5 (exigido por el curso)	P7 — Sprint 4
HU-05	Lector QR por cámara	Could Have / Baja	Valor: 3, Esfuerzo: 5 (alternativa: digitar placa)	P8 — Sprint 3
HU-09	Landing page	Could Have / Baja	Valor: 2, Esfuerzo: 3	P9 — Sprint 1 (ejercicio de frontend)

9. Cronograma de Sprints

El proyecto se organiza en cuatro sprints de dos semanas, del 30/09 al 25/11/2026, según el tablero de Sprints en Notion. Los estados corresponden a los registrados en Notion y al avance verificado en el repositorio de GitHub al 30/10/2026.

9.1. Resumen de Sprints

Sprint	Fechas	Objetivo del Sprint	Historias Asignadas	Story Points	Estado
Sprint 1	30/09 – 14/10	Arquitectura base, autenticación y turnos de caja	HU-01, HU-02, HU-09	16 SP	En proceso (vencido; 3 de 16 SP)
Sprint 2	14/10 – 28/10	Registro operativo, tarifas y tickets QR	HU-03, HU-04	16 SP	Sin iniciar
Sprint 3	28/10 – 11/11	Control de aforo y lector QR por cámara	HU-05, HU-06	13 SP	Sin iniciar
Sprint 4	11/11 – 25/11	Panel administrativo, métricas y despliegue	HU-07, HU-08	13 SP	Sin iniciar

9.2. Detalle por Sprint

Sprint 1

Objetivo: Disponer de la infraestructura base, el control de acceso por roles y la gestión de apertura y cierre de turnos de caja.

ID	Historia de Usuario	Responsable	Story Points	Estado
HU-01	Autenticación y control de acceso por roles	Nicolás Agudelo	8	En proceso
HU-02	Gestión de turnos de caja (Cash Shifts)	José García	5	Pendiente
HU-09	Landing page informativa de GoPark (HTML/CSS)	Nicolás Agudelo	3	Completada

Impedimentos encontrados: Se creó el repositorio (30/09), se construyó el backlog en Notion y se terminó la landing page (6 commits entre el 07/10 y el 11/10). El equipo la usó para nivelar conocimientos de frontend, pero HU-01 y HU-02 no llegaron a Done y el sprint sigue abierto en Notion después de su fecha de cierre (14/10). Causas: la curva de aprendizaje del stack (Next.js, NestJS, Prisma), la coincidencia con entregas de otras asignaturas y que se priorizó la landing sobre la base técnica. Acción: cerrar el Sprint 1 con su Review y su Retrospectiva y pasar HU-01 y HU-02 al sprint en curso con prioridad máxima.

Sprint 2

Objetivo: Implementar el flujo central de entrada y salida vehicular con cálculo de cobro en tiempo real.

ID	Historia de Usuario	Responsable	Story Points	Estado
HU-03	Registro de entrada y emisión de ticket QR	Dilan García	8	Sin iniciar
HU-04	Cálculo de tarifas y registro de salida	Nicolás Agudelo	8	Sin iniciar

Impedimentos encontrados: El sprint no se inició porque el Sprint 1 seguía abierto; además, HU-03 y HU-04 dependen de HU-01 y HU-02. Sus fechas (14/10 – 28/10) ya vencieron. Acción: se acordó trabajar con una rama por historia y Pull Requests revisados.

Sprint 3

Objetivo: Añadir agilidad operativa con el escaneo por cámara y proteger el cupo disponible ante accesos concurrentes.

ID	Historia de Usuario	Responsable	Story Points	Estado
HU-05	Escaneo de ticket QR mediante cámara web	José García	5	Sin iniciar
HU-06	Control de capacidad y concurrencia de cupos	Nicolás Agudelo	8	Sin iniciar

Impedimentos encontrados: Según el calendario es el sprint vigente (28/10 – 11/11), pero aún figura sin iniciar. El riesgo principal es la acumulación de historias pendientes (HU-01 a HU-04). Para mitigarlo, HU-05 (Could have) puede pasar al Sprint 4 si se necesita capacidad.

Sprint 4

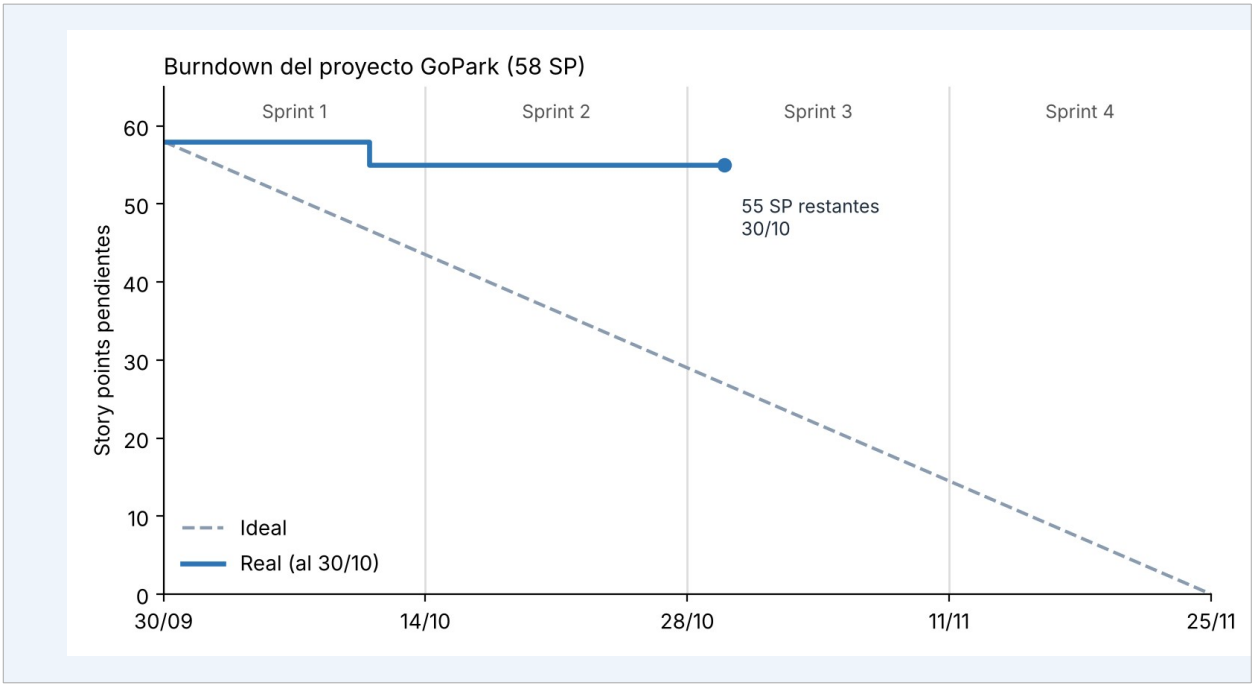
Objetivo: Proporcionar al administrador visibilidad del negocio y dejar el proyecto desplegado, probado y documentado.

ID	Historia de Usuario	Responsable	Story Points	Estado
HU-07	Dashboard de ocupación e ingresos	Dilan García	8	Sin iniciar
HU-08	Documentación técnica, CI/CD y despliegue	José García	5	Sin iniciar

Impedimentos encontrados: Sprint aún no iniciado.

9.3. Burndown Chart

El burndown se construyó a partir del backlog (58 SP) y del historial de commits del repositorio. La línea ideal supone un avance lineal hasta el 25/11; la línea real solo desciende con la historia cerrada (HU-09, 3 SP), por lo que al 30/10 quedan 55 SP. La brecha confirma el retraso y justifica la replanificación descrita.


Burndown del proyecto GoPark (58 SP)
Sprint 1
Sprint 2
Sprint 3
Sprint 4
Story points pendientes
60
50
40
30
20
10
0
55 SP restantes
30/10
Ideal
Real (al 30/10)
30/09
14/10
28/10
11/11
25/11

10. Conclusiones y Lecciones Aprendidas

10.1. Conclusiones

Conclusiones del primer avance, en relación con los Resultados de Aprendizaje del curso:

- **RAA1 — Arquitectura:** el equipo definió una arquitectura de monolito modular documentada con C4 (niveles 1 y 2) y decisiones justificadas (REST, PostgreSQL, bloqueo pesimista, Strategy). La implementación sobre esa arquitectura es la prioridad del Sprint 3.
- **RAA2 — Pruebas:** están identificados los casos críticos que se probarán: casos borde del motor de tarifas con Jest y concurrencia sobre el último cupo con una prueba de carga. Aún no se han ejecutado porque los módulos no existen.
- **RAA3 — CI/CD:** el pipeline de GitHub Actions y el despliegue en Vercel, Render y Supabase están diseñados (diagrama de despliegue); su configuración se adelantará al Sprint 3 para que cada Pull Request se valide desde el inicio.
- **Gestión Scrum:** el backlog, la priorización MoSCoW y el burndown dejaron ver a tiempo un retraso de dos sprints. El valor de esta entrega es que el equipo lo detectó con datos y ajustó el plan.

10.2. Lecciones Aprendidas

Reflexión del equipo sobre el proceso de desarrollo a mitad del semestre.

Aspecto	Lección Aprendida	Acción Futura
Trabajo en equipo	Los commits se concentraron en un solo integrante; el trabajo no quedó visible en el repositorio.	Una rama por historia y un Pull Request revisado por otro integrante antes de cada merge.
Gestión del tiempo	Se planificaron 13–16 SP por sprint sin descontar la carga de otras materias; el Sprint 1 venció sin cerrar sus historias principales.	Estimar con la capacidad real y mantener las dailies dos veces por semana para detectar bloqueos pronto.
Decisiones técnicas	Empezar por la landing page antes que por la base técnica retrasó las historias que dependían de ella.	Respetar las dependencias del backlog: autenticación y turnos primero.
Uso de herramientas	Notion organizó bien el backlog, pero no estaba conectado con el código.	Enlazar cada historia de Notion con su rama y su Pull Request en GitHub y activar GitHub Actions desde ya.

Referencias

Fuentes citadas en el documento, en formato APA 7.ª edición.

Alcaldía de Santiago de Cali. (2024a, 3 de septiembre). *Operativos de control vial: la apuesta de la Alcaldía de Cali para disminuir los altos índices de accidentalidad*. https://www.cali.gov.co/publicaciones/182713/

Alcaldía de Santiago de Cali. (2024b, 19 de septiembre). *Secretaría de Movilidad de Cali inicia la caracterización de parqueaderos fuera de la vía pública*. https://www.cali.gov.co/publicaciones/182932/

Bass, L., Clements, P. y Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley Professional.

Brown, S. (s.f.). *The C4 model for visualising software architecture*. https://c4model.com

Clegg, D. y Barker, R. (1994). *Case method fast-track: A RAD approach*. Addison-Wesley.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley Professional.

Humble, J. y Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley Professional.

Ley 1480 de 2011. (2011, 12 de octubre). *Por medio de la cual se expide el Estatuto del Consumidor y se dictan otras disposiciones*. Congreso de la República de Colombia.

Ley 1801 de 2016. (2016, 29 de julio). *Por la cual se expide el Código Nacional de Seguridad y Convivencia Ciudadana*. Congreso de la República de Colombia.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.ª ed.). O'Reilly Media.

OWASP Foundation. (s.f.). *Session management cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

PostgreSQL Global Development Group. (s.f.). *Explicit locking*. En PostgreSQL documentation. https://www.postgresql.org/docs/current/explicit-locking.html

Schwaber, K. y Sutherland, J. (2020). *La Guía de Scrum: La guía definitiva de Scrum: Las reglas del juego*. https://scrumguides.org

11. Anexos

Anexo A — Pantallas de Notion

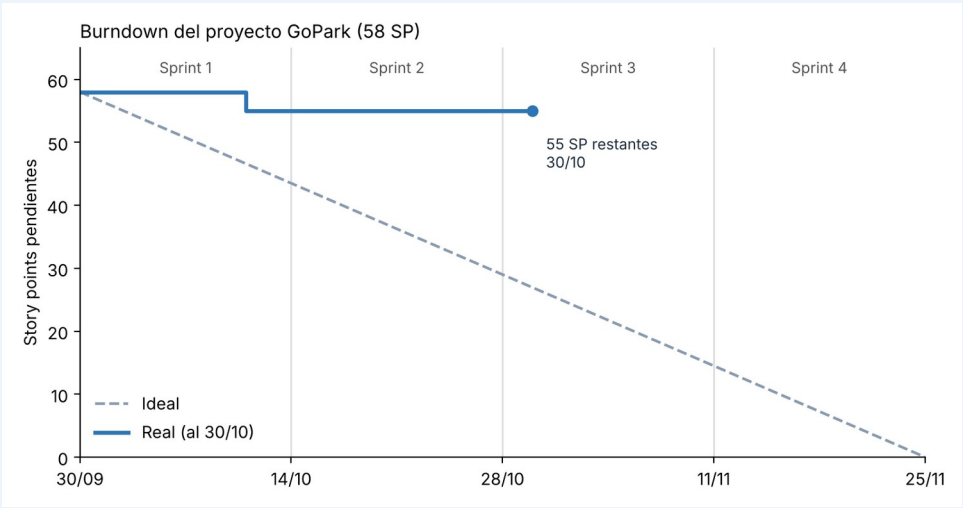
El equipo gestiona Scrum en Notion en lugar de Jira. Se incluyen capturas del Product Backlog, la base de datos de Sprints y el burndown.

[Captura A.1 — Product Backlog en Notion]

✎ *Descripción: Product Backlog en Notion con historias priorizadas. Fecha: 30/10/2026*



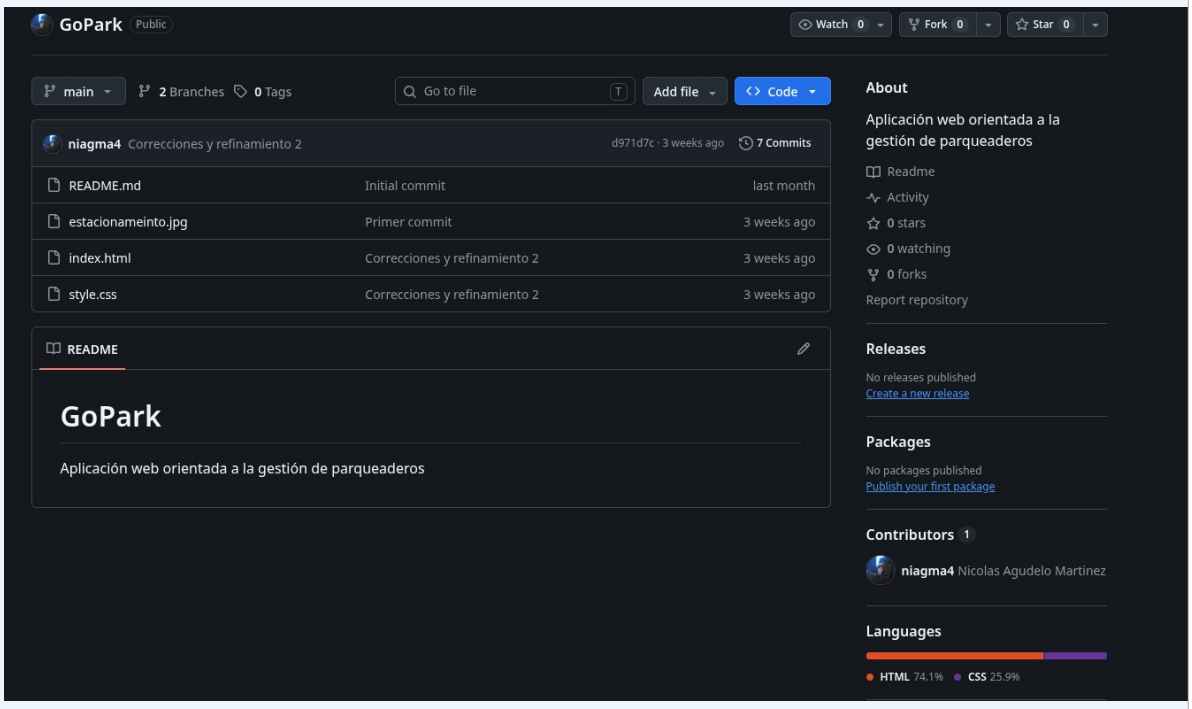
✎ *Descripción: Base de datos «Sprints» en Notion con estado, duración e historias de usuario de cada sprint. Fecha: 30/10/2026*



✎ *Descripción: Burndown del proyecto (ver figura de la sección 9.3). Fecha: 30/10/2026*

Anexo B — Pantallas de GitHub

Evidencia del repositorio https://github.com/niagma4/GoPark: vista general, historial de commits y ramas.



✎ *Descripción: Vista del repositorio con README, estructura de carpetas, 2 ramas y 7 commits. Fecha: 30/10/2026*

```
$ git log --all --graph --date=short --format='%h %ad %an %s'
* d971d7c 2026-10-11 niagma4  Correcciones y refinamiento 2
* 750c8b2 2026-10-09 niagma4  Corrigiendo malas practicas - 1
* 55aeb64 2026-10-09 niagma4  Correcciones y refinamiento 1
* dab6ffa 2026-10-08 niagma4  Agregando colores y varios
* 5b669d8 2026-10-07 niagma4  Aprendiendo CSS
* 2806070 2026-10-07 niagma4  Primer commit
* 5dfee7d 2026-09-30 Nicolas Agudelo Martinez  Initial commit

$ git branch -a
* main
  remotes/origin/CSS
  remotes/origin/main
```

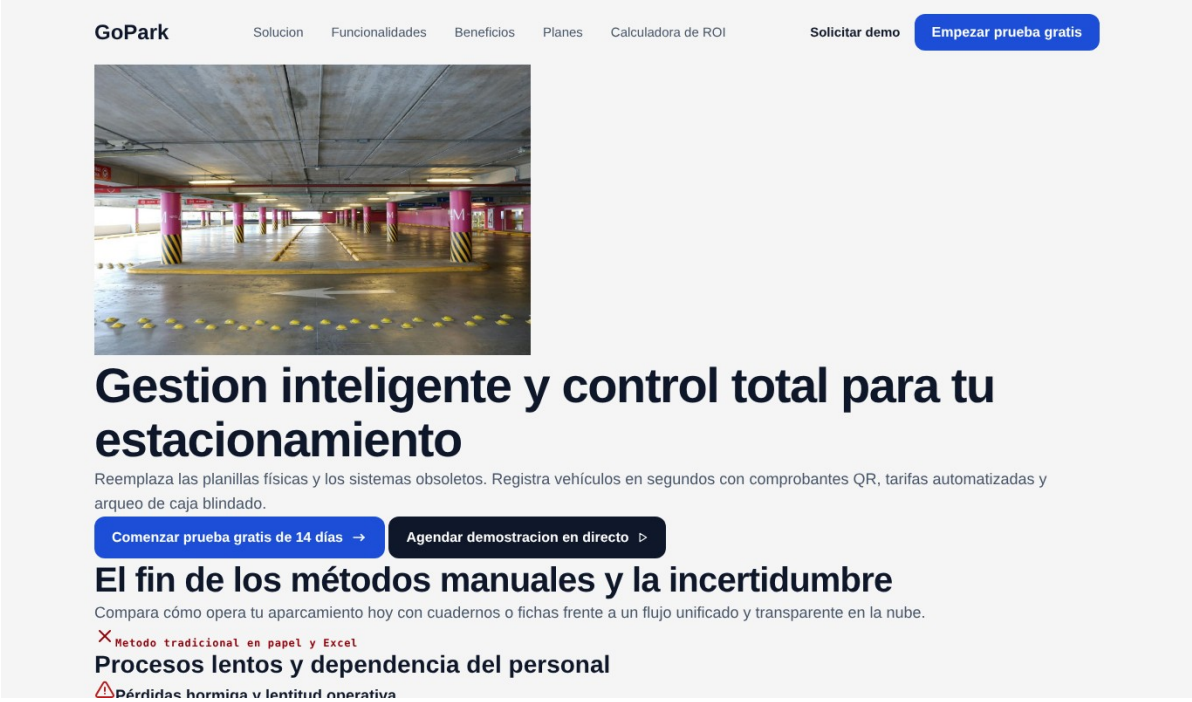
✎ *Descripción: Historial de commits y ramas (main y CSS) obtenido con git log. Fecha: 30/10/2026*

[Captura B.3 — Pull Requests y Ramas]

✎ *Descripción: Ramas del proyecto (main, CSS). Aún no hay Pull Requests; se usarán a partir del Sprint 3. Fecha: 30/10/2026*

Anexo C — Pantallas del Sistema Funcional

Estado actual del sistema: landing page informativa de GoPark (HU-09), ejecutada localmente desde el repositorio.



✎ *Descripción: Captura C.1 — Sección principal de la landing page de GoPark. Fecha: 30/10/2026*

— Fin del documento —